

Generic SortedList<TKey, TValue> Collection Class in C# with Examples

In this article, I am going to discuss the **Generic SortedList<TKey, TValue> Collection Class in C#** with Examples. Please read our previous article where we discussed the [Generic HashSet<T> Collection Class in C#](#) with Examples. The Generic SortedList<TKey, TValue> collection class belongs to System.Collections.Generic namespace. At the end of this article, you will understand the following pointers with examples.

1. [What is SortedList<TKey, TValue> in C#?](#)
2. [Methods, Properties, and Constructor of Generic SortedList<TKey, TValue> Collection Class in C#](#)
3. [How to Create a SortedList<TKey, TValue> Collection in C#?](#)
4. [How to Add Elements into a Generic SortedList Collection in C#?](#)
5. [How to Access a Generic SortedList Collection in C#?](#)
6. [How to Remove Elements from a Generic SortedList Collection in C#?](#)
7. [How to check the Availability of key/value Pairs in a Generic SortedList Collection in C#?](#)
8. [Generic SortedList Collection with Complex Type in C#](#)
9. [What are the differences between SortedList<TKey, TValue> and SortedDictionary<TKey, TValue> in C#?](#)
10. [When to use Generic SortedList Collection?](#)

What is SortedList<TKey, TValue> in C#?

The Generic SortedList<TKey, TValue> Collection Class in C# represents a collection of key/value pairs that are sorted by the keys based on the associated IComparer implementation. By default, it sorts the key/value pairs in ascending order. For example, if the keys are of primitive data types, then it sorted the collection in ascending order of keys, and if the keys are of string type, then it will sort the element alphabetically.

With the help of a key, we can easily search for or remove elements from the SortedList collection. The generic SortedList collection can be accessible either by using the keys or by using the integral index.

In C#, the SortedList<TKey, TValue> Collection allows us to store duplicate values, but the keys must be unique and cannot be null. The size of SortedList will vary dynamically so that you can add or remove elements from the SortedList collection based on your application requirements.

Methods, Properties, and Constructor of Generic SortedList<TKey, TValue> Collection Class in C#:

If you go to the definition of Generic SortedList<TKey, TValue> Collection Class, then you will see the following.

```

public class SortedList<TKey, TValue> : ICollection<KeyValuePair<TKey, TValue>>, IEnumerable<KeyValuePair<TKey, TValue>>
{
    ...public SortedList();
    ...public SortedList(IComparer<TKey>? comparer);
    ...public SortedList(IDictionary<TKey, TValue> dictionary);
    ...public SortedList(int capacity);
    ...public SortedList(IDictionary<TKey, TValue> dictionary, IComparer<TKey>? comparer);
    ...public SortedList(int capacity, IComparer<TKey>? comparer);

    ...public TValue this[TKey key] { get; }
    ...public int Count { get; }
    ...public IComparer<TKey> Comparer { get; }
    ...public int Capacity { get; set; }
    ...public IList<TKey> Keys { get; }
    ...public IList<TValue> Values { get; }

    ...public void Add(TKey key, TValue value);
    ...public void Clear();
    ...public bool ContainsKey(TKey key);
    ...public bool ContainsValue(TValue value);
    ...public IEnumerator<KeyValuePair<TKey, TValue>> GetEnumerator();
    ...public int IndexOfKey(TKey key);
    ...public int IndexOfValue(TValue value);
    ...public bool Remove(TKey key);
    ...public void RemoveAt(int index);
    ...public void TrimExcess();
    ...public bool TryGetValue(TKey key, [MaybeNullWhen(false)] out TValue value);
}

```

How to Create a SortedList<TKey, TValue> collection in C#?

The Generic SortedList<TKey, TValue> Collection class in C# provided the following six constructors that we can use to create an instance of SortedList.

1. **SortedList():** It initializes a new instance of the System.Collections.Generic.SortedList class that is empty, has the default initial capacity, and uses the default System.Collections.Generic.IComparer
2. **SortedList(IComparer<TKey> comparer):** It initializes a new instance of the System.Collections.Generic.SortedList class that is empty, has the default initial capacity and uses the specified System.Collections.Generic.IComparer. The Parameter comparer specifies the System.Collections.Generic.IComparer implementation to use when comparing keys. -or- null to use the System.Collections.Generic.Compare for the type of the key.
3. **SortedList(IDictionary<TKey, TValue> dictionary):** It initializes a new instance of the System.Collections.Generic.SortedList class that contains elements copied from the specified System.Collections.Generic.IDictionary, has sufficient capacity to accommodate the number of elements copied and uses the default System.Collections.Generic.IComparer. The Parameter dictionary specifies the System.Collections.Generic.IDictionary whose elements are copied to the new System.Collections.Generic.SortedList.
4. **SortedList(int capacity):** It initializes a new instance of the System.Collections.Generic.SortedList class that is empty, has the specified initial capacity and uses the default System.Collections.Generic.IComparer. The Parameter capacity specifies the initial number of elements that the System.Collections.Generic.SortedList can contain.
5. **public SortedList(IDictionary<TKey, TValue> dictionary, IComparer<TKey>? comparer):** It initializes a new instance of the System.Collections.Generic.SortedList class that contains elements copied from the specified System.Collections.Generic.IDictionary, has sufficient capacity to accommodate the number of elements copied and uses the specified System.Collections.Generic.IComparer. The Parameter comparer specifies the System.Collections.Generic.IComparer implementation to use when comparing keys. -or- null to use the default System.Collections.Generic.Comparer for the type of the key. The Parameter dictionary

specifies the System.Collections.Generic.IDictionary whose elements are copied to the new System.Collections.Generic.SortedList.

6. **SortedList(int capacity, IComparer<TKey>? comparer):** It initializes a new instance of the System.Collections.Generic.SortedList class that is empty, has the specified initial capacity and uses the specified System.Collections.Generic.IComparer. The Parameter comparer specifies the System.Collections.Generic.IComparer implementation to use when comparing keys. -or- null to use the default System.Collections.Generic.Comparer for the type of the key. The Parameter capacity specifies the initial number of elements that the System.Collections.Generic.SortedList can contain.

Let's see how to create a SortedList<TKey, TValue> collection using the SortedList() constructor in C#:

Step1:

As the SortedList<TKey, TValue> Collection Class belongs to System.Collections.Generic namespace, so first, we need to import the System.Collections.Generic namespace into our program as follows:

using System.Collections.Generic;

Step2:

Next, we need to create an instance of the SortedList<TKey, TValue> collection class using the SortedList() constructor as follows:

SortedList<TKey, TValue> sortedListName = new SortedList<TKey, TValue>();

How to Add Elements into a Generic SortedList Collection in C#?

If you want to add a key/value pair to a Generic SortedList Collection, then you need to use the Add() method provided by the Generic SortedList class.

- **Add(TKey key, TValue value):** The Add(TKey key, TValue value) method is used to add an element with the specified key and value into the Generic SortedList. Here, the parameter key specifies the key of the element to add and the parameter value specifies the element to add. The value can be null for reference types. If the key is null, then it will throw ArgumentNullException and if an element with the same key already exists in the Generic SortedList, then it will throw ArgumentException.

For example, here we are creating a generic SortedList by specifying the key as an integer and the value as a string.

```
SortedList<int, string> genericSortedList = new SortedList<int, string>();  
genericSortedList.Add(1, "One");  
genericSortedList.Add(3, "Three");  
genericSortedList.Add(2, "Two");
```

The following statement will give us a compile-time error as we pass the key as null.

```
genericSortedList.Add(null, "Four");
```

The following statement will throw ArgumentException saying **An entry with the same key already exists** as we pass the key as 2 which already exists in the collection.

```
genericSortedList.Add(2, "AnyValue");
```

As this class has the Add method, so, you can also store the key/value pairs in the Generic SortedList Collection using Collection_INITIALIZER as follows.

```
SortedList<int, string> genericSortedList = new SortedList<int, string>  
{  
    { 1, "One"},  
    { 3, "Three"},  
    { 2, "Two"}  
};
```

How to Access a Generic SortedList Collection in C#?

We can access the key/value pairs of the Generic SortedList Collection in C# using three different ways. They are as follows:

Using for loop to access Generic SortedList Collection in C#:

As the SortedList Collection has the Indexer, so, you can use a for loop to access the key/value pairs as shown below.

```
for (int i = 0; i < genericSortedList.Count; i++)
{
    Console.WriteLine("Keys : " + genericSortedList.Keys[i] + "\tValues : " +
genericSortedList.Values[i]);
}
```

Using Index to Access Generic SortedList Collection in C#:

You can access the individual value of the Generic SortedList Collection in C# by using the index. In this case, we need to pass the key as a parameter to find the respective value. For example,

```
Console.WriteLine($"Value of Key 2 is:{ genericSortedList[2]}");
string val = (string)genericSortedList[3];
Console.WriteLine(val);
```

Using for-each loop to access Generic SortedList Collection in C#:

You can also use a for-each loop to access the key/value pairs of the Generic SortedList Collection in C# as follows.

```
foreach (KeyValuePair<int, string> item in genericSortedList)
{
    Console.WriteLine($"Key: { item.Key}, Value: { item.Value}");
}
```

Note: You can add elements to the SortedList collection in random order. But they are going to be stored in the collection in Ascending order based on key.

Example to Understand How to Create a Generic SortedList Collection and Add Elements in C#:

For a better understanding of how to create a Generic SortedList Collection and how to add elements to the collection, and how to access the elements from the collection, please have a look at the below example.

```
using System;
using System.Collections.Generic;
namespace GenericSortedListDemo
{
    class Program
    {
        static void Main()
        {
```

```

//Creating Generic SortedList Collection
SortedList<int, string> genericSortedList = new SortedList<int, string>()
//Adding Elements to SortedList Collection using Add Method in Rar
genericSortedList.Add(1, "One");
genericSortedList.Add(5, "Five");
genericSortedList.Add(4, "Four");
genericSortedList.Add(2, "Two");
genericSortedList.Add(3, "Three");
//You cannot pass null as a key
//genericSortedList.Add(null, "Ten"); //Compile-Time Error
//Duplicate Key not allowed
//System.ArgumentException: 'An entry with the same key already exists'
//genericSortedList.Add(2, "Any Value");
//Accessing Generic SortedList Collection using For loop
Console.WriteLine("Accessing Generic SortedList using For loop");
for (int i = 0; i < genericSortedList.Count; i++)
{
    Console.WriteLine("Keys : " + genericSortedList.Keys[i] + "\t\t" +
genericSortedList.Values[i]);
}
//Accessing Generic SortedList Collection using For Each loop
Console.WriteLine("\nAccessing SortedList using For Each loop");
foreach (KeyValuePair<int, string> item in genericSortedList)
{
    Console.WriteLine($"Key: { item.Key}, Value: { item.Value}");
}
//Accessing Generic SortedList Individual Elements using Keys
Console.WriteLine("\nAccessing SortedList Individual Elements using Keys");
Console.WriteLine($"Key: 1, Value: {genericSortedList[1]}");
Console.WriteLine($"Key: 2, Value: {genericSortedList[2]}");
Console.WriteLine($"Key: 3, Value: {genericSortedList[3]}");
Console.ReadKey();
}
}
}

```

Output:

```

Accessing Generic SortedList using For loop
Keys : 1      Values : One
Keys : 2      Values : Two
Keys : 3      Values : Three
Keys : 4      Values : Four
Keys : 5      Values : Five

Accessing SortedList using For Each loop
Key: 1, Value: One
Key: 2, Value: Two
Key: 3, Value: Three
Key: 4, Value: Four
Key: 5, Value: Five

Accessing SortedList Individual Elements using Keys
Key: 1, Value: One
Key: 2, Value: Two
Key: 3, Value: Three

```

Please note here we are getting the output based on the ascending order of the keys when accessing the elements either using for loop or using For Each loop.

Adding Elements to Generic SortedList Collection using Collection Initializer in C#:

As the Generic SortedList class has the Add method which takes TKey and TValue as parameters, so we can use the collection initializer to initialize the collection while creating. In the following example, we are using Collection Initializer syntax instead of the Add method to add key-value pairs into the Generic SortedList Collection in C#.

```

using System;
using System.Collections.Generic;
namespace GenericSortedListDemo
{
    class Program
    {
        static void Main()
        {
            //Creating Generic SortedList Collection using Collection Initializer
            SortedList<int, string> genericSortedList = new SortedList<int, string>
            {
                { 1, "One" },
                { 5, "Five" },
                { 4, "Four" },
                { 2, "Two" },
                { 3, "Three" }
            };
            //Accessing Generic SortedList Collection using For Each loop
            Console.WriteLine("Accessing SortedList using For Each loop");
            foreach (KeyValuePair<int, string> item in genericSortedList)

```

```

        {
            Console.WriteLine($"Key: { item.Key}, Value: { item.Value}");
        }
        Console.ReadKey();
    }
}

```

Output:

```

Accessing SortedList using For Each loop
Key: 1, Value: One
Key: 2, Value: Two
Key: 3, Value: Three
Key: 4, Value: Four
Key: 5, Value: Five

```

How to Remove Elements from a Generic SortedList Collection in C#?

The Generic SortedList Collection Class in C# provides the following methods to remove elements from SortedList.

1. **Remove(TKey key):** This method is used to remove the element with the specified key from the System.Collections.Generic.SortedList. The parameter key specifies the element to remove. It returns true if the element is successfully removed; otherwise, false. This method also returns false if the key was not found in the original Generic SortedList.
2. **RemoveAt(int index):** This method is used to remove the element at the specified index of a Generic SortedList. The parameter index specifies the element to remove. It is 0 based Index.
3. **Clear():** This method is used to remove all elements from a Generic SortedList Collection.

Let us see an example to understand the above three methods of Generic SortedList Collection Class in C#. Please have a look at the below example.

```

using System;
using System.Collections.Generic;
namespace GenericSortedListDemo
{
    class Program
    {
        static void Main()
        {
            //Creating a Generic SortedList Collection Object
            //Here both the keys and values are of string type
            SortedList<string, string> genericSortedList = new SortedList<stri
();

            //Adding Elements to Generic SortedList Collection using Add Metho
            genericSortedList.Add("Ind", "India");
            genericSortedList.Add("USA", "United State of America");
            genericSortedList.Add("SA", "South Africa");

```

```

        genericSortedList.Add("PAK", "Pakistan");
        //Accessing Generic SortedList Collection using For Each loop
        Console.WriteLine($"SortedList Elements Count={genericSortedList.Count}, Capacity:{genericSortedList.Capacity}");
        foreach (KeyValuePair<string, string> item in genericSortedList)
        {
            Console.WriteLine($"Key: { item.Key}, Value: { item.Value}");
        }
        // Remove value having key PAK Using Remove() method
        genericSortedList.Remove("PAK");
        Console.WriteLine($"SortedList Elements After Remove Method Count={genericSortedList.Count}, Capacity:{genericSortedList.Capacity}");
        foreach (KeyValuePair<string, string> item in genericSortedList)
        {
            Console.WriteLine($"Key: { item.Key}, Value: { item.Value}");
        }
        // Remove element at index 1 Using RemoveAt() method
        genericSortedList.RemoveAt(1);
        Console.WriteLine($"SortedList Elements After RemoveAT Method Count={genericSortedList.Count}, Capacity:{genericSortedList.Capacity}");
        foreach (KeyValuePair<string, string> item in genericSortedList)
        {
            Console.WriteLine($"Key: { item.Key}, Value: { item.Value}");
        }
        // Remove all key/value pairs Using Clear method
        genericSortedList.Clear();
        Console.WriteLine($"After Clear Method Count={genericSortedList.Count}, Capacity:{genericSortedList.Capacity}");
        Console.ReadKey();
    }
}

```

Output:

```

SortedList Elements Count=4, Capacity:4
Key: Ind, Value: India
Key: PAK, Value: Pakistan
Key: SA, Value: South Africa
Key: USA, Value: United State of America

SortedList Elements After Remove Method Count=3, Capacity:4
Key: Ind, Value: India
Key: SA, Value: South Africa
Key: USA, Value: United State of America

SortedList Elements After RemoveAT Method Count=2, Capacity:4
Key: Ind, Value: India
Key: USA, Value: United State of America
After Clear Method Count=0, Capacity:4

```

How to check the Availability of key/value Pairs in a Generic SortedList Collection in C#?

If you want to check whether the key/value pair exists or not in the Generic SortedList Collection in C#, then you can use the following methods provided by the Generic SortedList class as per your requirement i.e. whether you want to search for an element by key or an element by value.

1. **ContainsKey(TKey key):** This method is used to determine whether the Generic SortedList collection contains a specific key. The parameter key to locating in the Generic SortedList object. It returns true if the Generic SortedList collection contains an element with the specified key; otherwise, false. If the key is null, then it will throw System.ArgumentNullException.
2. **ContainsValue(TValue value):** This method is used to determine whether a Generic SortedList contains a specific value. The parameter value to locate in the Generic SortedList collection. The value can be null for reference types. It returns true if the Generic SortedList Collection contains an element with the specified value; otherwise, false.

Let us understand this with an example. The following example shows how to use the both ContainsKey and ContainsValue method of the Generic SortedList collection class in C#.

```

using System;
using System.Collections.Generic;
namespace GenericSortedListDemo
{
    class Program
    {
        static void Main()
        {
            //Creating a Generic SortedList Collection Object
            //Here both the keys and values are of string type
            SortedList<string, string> genericSortedList = new SortedList<stri
            {
                { "Ind", "India" },
                { "USA", "United State of America" },
            }

```

```

        { "SA", "South Africa" },
        { "PAK", "Pakistan" }
    };
    //Accessing Generic SortedList Collection using For Each loop
    Console.WriteLine("Generic SortedList Elements");
    foreach (KeyValuePair<string, string> item in genericSortedList)
    {
        Console.WriteLine($"Key: { item.Key}, Value: { item.Value}");
    }
    //Checking the key using the ContainsKey method
    Console.WriteLine("\nIs Ind Key Exists : " +
genericSortedList.ContainsKey("Ind"));
    Console.WriteLine("Is NZ Key Exists : " +
genericSortedList.ContainsKey("NZ"));
    //Checking the value using the ContainsValue method
    Console.WriteLine("\nIs India value Exists : " +
genericSortedList.ContainsValue("India"));
    Console.WriteLine("Is Bangladesh value Exists : " +
genericSortedList.ContainsValue("Bangladesh"));
    Console.ReadKey();
    }
}
}

```

Output:

```

Generic SortedList Elements
Key: Ind, Value: India
Key: PAK, Value: Pakistan
Key: SA, Value: South Africa
Key: USA, Value: United State of America

Is Ind Key Exists : True
Is NZ Key Exists : False

Is India value Exists : True
Is Bangladesh value Exists : False

```

Generic SortedList Collection with Complex Type in C#:

As of now, we have used the built-in data types type with SortedList Class. Now, let us proceed further and see how to create a Generic SortedList collection of Complex types. Let us create a class called Student and then let us create a collection of Student types and also add duplicate elements.

```

using System;
using System.Collections.Generic;
namespace GenericsDemo
{
    class Program

```

```

{
    static void Main()
    {
        //Here we are creating a sortedlist whose key is int and value is
        SortedList<int, Student> genericSortedList = new SortedList<int, Student>
        {
            { 101, new Student(){ ID = 101, Name = "Anurag", Branch="CSE"}},
            { 102, new Student(){ ID = 102, Name = "Mohanty", Branch="CSE"}},
            { 103, new Student(){ ID = 103, Name = "Sambit", Branch="ETC"}}
        };
        //The following Statement will give you Runtime Exception as the key already exists
        //genericSortedList.Add(101, new Student() { ID = 101, Name = "Anurag", Branch = "CSE" });
        //Accessing Generic SortedList Collection using For Each loop
        Console.WriteLine("Generic SortedList Elements");
        foreach (KeyValuePair<int, Student> item in genericSortedList)
        {
            Console.WriteLine($"Key: { item.Key}: ID: { item.Value.ID}, Name: { item.Value.Name}, Branch: { item.Value.Branch}");
        }
        Console.ReadKey();
    }
}

public class Student
{
    public int ID { get; set; }
    public string Name { get; set; }
    public string Branch { get; set; }
}

```

Output:

```

Generic SortedList Elements
Key: 101: ID: 101, Name: Anurag, Branch: CSE
Key: 102: ID: 102, Name: Mohanty, Branch: CSE
Key: 103: ID: 103, Name: Sambit, Branch: ETC

```

Properties of Generic SortedList Collection Class in C#:

1. **Count:** It returns the number of key/value pairs contained in the Generic SortedList collection.
2. **Comparer:** It returns the System.IComparable to the current Generic SortedList object.
3. **Capacity:** It is used to get or set the number of elements that the Generic SortedList can contain. It returns the number of elements that the Generic SortedList can contain.
4. **Keys:** It returns a collection containing the keys in the Generic SortedList in sorted order.
5. **Values:** It returns a collection containing the values in the Generic SortedList.

Example to Understand Generic SortedList Collection Class Properties in C#

```
using System;
using System.Collections.Generic;
namespace GenericsDemo
{
    class Program
    {
        static void Main()
        {
            SortedList<string, string> genericSortedList = new SortedList<string, string>
            {
                { "Ind", "India" },
                { "USA", "United State of America" },
                { "SA", "South Africa" },
                { "PAK", "Pakistan" }
            };
            Console.WriteLine($"Count Property : {genericSortedList.Count}");
            Console.WriteLine($"Comparer Property : {genericSortedList.Comparer}");
            Console.WriteLine($"Capacity Property : {genericSortedList.Capacity}");
            Console.WriteLine("\nKeys Property");
            foreach (var item in genericSortedList.Keys)
            {
                Console.WriteLine(item);
            }
            Console.WriteLine("\nValues Property");
            foreach (var item in genericSortedList.Values)
            {
                Console.WriteLine(item);
            }
            Console.ReadKey();
        }
    }
}
```

Output:

```
Count Property : 4
Comparer Property : System.Collections.Generic.GenericComparer`1[System.String]
Capacity Property : 4

Keys Property
Ind
PAK
SA
USA

Values Property
India
Pakistan
South Africa
United State of America
```

What are the differences between SortedList<TKey, TValue> and SortedDictionary<TKey, TValue> in C#?

The differences between SortedList<TKey, TValue> and SortedDictionary<TKey, TValue> in C# are as follows:

1. The SortedList<TKey, TValue> collection uses less memory than SortedDictionary<TKey, TValue> collection.
2. The SortedDictionary<TKey, TValue> collection has faster insertion and removal operations for unsorted data compared to the SortedList<TKey, TValue> collection.
3. If the elements are populated all at once from sorted data then SortedList<TKey, TValue> collection works faster than SortedDictionary<TKey, TValue> collection.
4. The SortedList supports efficient indexed retrieval of keys and values which is not supported by SortedDictionary.
5. SortedList<TKey, TValue> collection doesn't allow inserting null and duplicate values for the keys. Every key in a SortedList must be unique otherwise it throws ArgumentException.

Note: The C# Programming Language supports both generic and non-generic SortedList collections. It is recommended to use generic SortedList<TKey, TValue> because it performs faster and is less error-prone compared to the non-generic SortedList.

Generic SortedList<TKey, TValue> Collection Class Summary:

The following are the important points that need to remember about the sorted list in c#.

1. The Generic SortedList Collection is used to store a collection of key/value pairs sorted by key based on the associated IComparer<T> implementation.
2. The Generic SortedList Collection allows us to store duplicate values, but keys must be unique to identify the values in the sorted list. Further, the key cannot be null but the value can be null for a reference type.
3. You can access Generic SortedList Elements either by using keys or with for and for each loop. In the for-each loop, we need to use KeyValuePair<TKey, TValue> to get key/value pairs from SortedList.

In the next article, I am going to discuss the [Generic SortedSet<T> Collection Class in C#](#) with Examples. Here, in this article, I try to explain the **Generic SortedList<TKey, TValue> Collection Class in C#** with Examples. I hope this Generic SortedList<TKey, TValue> Collection Class in C# with Examples article will help you with your needs. I would like to have your feedback. Please post your feedback, question, or comments about this article.